

Bounded Controller Synthesis for CGBF Controller

Tintin Yuan

August 8, 2025

1 Problem formulation

Known condition:

- System dynamics:

$$\dot{\mathbf{x}} = f(\mathbf{x}) + g(\mathbf{x})\mathbf{u}, \quad (1)$$

let's say, Dubins car.

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \\ \dot{\theta} \\ \dot{v} \end{bmatrix} = \underbrace{\begin{bmatrix} v \cos \theta \\ v \sin \theta \\ 0 \\ 0 \end{bmatrix}}_{f(\mathbf{x})} + \underbrace{\begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}}_{g(\mathbf{x})} \underbrace{\begin{bmatrix} \omega \\ a \end{bmatrix}}_{\mathbf{u}}, \quad (2)$$

- Value function $h(\mathbf{x})$ that quantifies the safe region $h(\mathbf{x}) > 0$.

Constraints:

Bounded constraints on control inputs

$$a \leq \mathbf{u} = k(\mathbf{x}) \leq b. \quad (3)$$

Goal:

Find controller $k(\mathbf{x})$ s.t. satisfies the constraints in (3), and preserves the invariant set.

1.1 Assumptions

We assume that the desired region for $h(\mathbf{x}) > 0$ is a closed region. i.e. the value of $h(\mathbf{x})$ is bounded. It makes sense for real-world scenarios, for example, the distant constraint-based obstacle avoidance problem.

2 Methods

2.1 Method 1: Conventional CBF approach

Let $u = k(\mathbf{x})$, we need the system to ensure the following CBF constraint:

$$\dot{h} \geq -\alpha(h), \forall \mathbf{x} \in \{\mathbf{x} | h(\mathbf{x}) \geq 0\}, \quad \alpha \in \text{class-}\mathcal{K}_\infty, \quad (4)$$

which can be converted into the following SOS constraint:

$$\nabla h [f + g \cdot k(\mathbf{x})] + \alpha \circ h \in \sum [\mathbf{x}], \quad (5)$$

where $\sum [\mathbf{x}]$ is the sum-of-square polynomials over the variable x .

Ideally, if we want to further add constraints to control inputs, we can incorporate the following constraint:

$$\begin{cases} k(\mathbf{x}) - a - s_0 \cdot h \in \sum [\mathbf{x}] \\ b - k(\mathbf{x}) - s_1 \cdot h \in \sum [\mathbf{x}], \end{cases} \quad (6)$$

where $s_0, s_1 \in \sum [\mathbf{x}]$ are auxiliary SOS polynomials to constrain the satisfiability of (3) to a bounded region for boundedness.

However, if we want the output function to be a function of the position variables $h(x, y) : \mathbb{R}^2 \rightarrow \mathbb{R}$ of the Dubins car, the relative degree of the system will be 2. Therefore, time-invariant Higher-Order CBF (HO-CBF) is needed, where we define the auxiliary functions:

- $\psi_1 = \dot{h} + \alpha_1 \circ h \geq 0$
- $\psi_2 = \dot{\psi}_1 + \alpha_2 \circ \psi_1 \geq 0$

where, $\alpha_1, \alpha_2 \in \text{class-}\mathcal{K}_\infty$, which leads to the following constraints:

$$\begin{cases} L_f h + \alpha_1 \circ h \geq 0 \\ L_f^2 h + L_g L_f h \mathbf{u} + L_f(\alpha_1 \circ h) + \alpha_2 \circ (L_f h + \alpha_1 \circ h) \geq 0. \end{cases} \quad (7)$$

For simplicity, let's assume $\alpha_1, \alpha_2 \in \mathbb{R}^+$, then (7) simplifies to

$$\begin{cases} L_f h + \alpha_1 h \geq 0 \\ L_f^2 h + L_g L_f h \mathbf{u} + (\alpha_1 + \alpha_2) L_f h + \alpha_1 \alpha_2 h \geq 0. \end{cases} \quad (8)$$

2.1: Non-convex SDP trick

Since multiplying two decision variables (α_1, α_2) makes the problem non-convex, which is outside the scope of SDP and most convex solvers. And since we cannot simply reformulate (8) by substituting $(\alpha_1 + \alpha_2) \rightarrow a$ and $\alpha_1 \alpha_2 \rightarrow b$ with $a, b \in \mathbb{R}^+$ since α_1, a, b has an internal relationship $(a - \alpha_1) \cdot \alpha_1 = b$ that cannot be captured in linear form, thus it will be hard to solve the above problem via SOS solver.

2.2 Method 2: CGBF approach

Previously proposed CGBF can avoid using HO-CBF by using a coordinate transform and back-stepping approach, which enforces the following invariant set defined on the output function:

$$\Psi(\mathbf{x}) = \psi(\mathbf{y}(\mathbf{x})) - \sum_{i=1}^m \sum_{l=1}^{\gamma_i-1} \frac{1}{2\mu_l} \|\eta_{l+1}^i - k_l^i(\mathbf{z}_l^i)\|_2^2, \quad (9)$$

where

$$\boldsymbol{\eta}^i = \begin{bmatrix} \eta_1^i \\ \vdots \\ \eta_{r_i}^i \end{bmatrix} = \begin{bmatrix} y_i(\mathbf{x}) \\ \vdots \\ \mathcal{L}_f^{r_i-1} y_i(\mathbf{x}) \end{bmatrix} = \begin{bmatrix} h_i(\mathbf{x}) \\ \vdots \\ \mathcal{L}_f^{r_i-1} h_i(\mathbf{x}) \end{bmatrix} \in \mathbb{R}^{r_i}, \quad (10)$$

is the new coordinate transformed through the output function, and the controller $k(\mathbf{x})$ which ensures the invariant of (9) can be computed via:

$$\left\{ \begin{array}{l} \frac{\partial \psi(\mathbf{y}(\mathbf{x}))}{\partial \mathbf{y}} \cdot \mathbf{k}_1(\mathbf{y}(\mathbf{x})) \geq \lambda \psi(\mathbf{y}(\mathbf{x})), \forall \mathbf{x} \in \overline{\mathcal{C}} \setminus \mathcal{X}^r \\ \mathbf{k}_2^i(\mathbf{z}_2^i) = \mu_1 \frac{\partial \psi(\mathbf{y}(\mathbf{x}))}{\partial \mathbf{y}_i} + \sum_{s=1}^1 \frac{\partial k_1^i}{\partial \eta_s^i} \eta_{s+1}^i + \frac{\lambda}{2} (\eta_2^i - k_1^i) \\ b_i(\mathbf{x}) = -\mathcal{L}_f^{\gamma_i} h_i(\mathbf{x}) - \frac{\mu_{\gamma_i-1} (\eta_{\gamma_i-1}^i - k_{\gamma_i-2}^i)}{\mu_{\gamma_i-2}} \\ \quad + \sum_{s=1}^{\gamma_i-1} \frac{\partial k_{\gamma_i-1}^i}{\partial \eta_s^i} \eta_{s+1}^i + \frac{\lambda}{2} (\eta_{\gamma_i}^i - k_{\gamma_i-1}^i) \\ \lambda > 0, \mu_l > 0, \forall l \in \{1, \dots, \gamma_i - 1\}, \forall i \in \{1, \dots, m\}. \\ \mathbf{b}(\mathbf{x}) = [b_1(\mathbf{x}) \quad \dots \quad b_m(\mathbf{x})]^\top \\ \mathbf{A}(\mathbf{x}) \text{ is defined as in (12)} \\ \mathbf{k}(\mathbf{x}) = \mathbf{A}^{-1}(\mathbf{x}) \mathbf{b}(\mathbf{x}), \end{array} \right. \quad (11)$$

where $\mathbf{A}$ is the matrix derived via the feedback linearisation process for the coordinate transform:

$$\mathbf{A}(\mathbf{x}) = \begin{bmatrix} \mathcal{L}_{g_1} \mathcal{L}_f^{r_1-1} h_1(\mathbf{x}) & \dots & \mathcal{L}_{g_m} \mathcal{L}_f^{r_1-1} h_1(\mathbf{x}) \\ \vdots & \ddots & \vdots \\ \mathcal{L}_{g_1} \mathcal{L}_f^{r_m-1} h_m(\mathbf{x}) & \dots & \mathcal{L}_{g_m} \mathcal{L}_f^{r_m-1} h_m(\mathbf{x}) \end{bmatrix}, \quad (12)$$

and μ_l is a decision variable that determines the feasible region of the invariant set as well as the magnitude of the control inputs. Since the Dubins car is of relative degree 2, only μ_1 is needed, and we simplify it as μ .

Since $\mu \in \mathbb{R}^+$, if we make $\mu(\mathbf{x})$ a function of $\mathbf{x}$ as another SOS polynomial to be determined according to the input bounds rather than a scalar, then the controller $k(\mathbf{x})$ in (11) becomes $k(\mathbf{x}, \mu)$. Further, for every bounded input constraint (3), we can derive a relationship $\mu(\mathbf{x}) \in \sum[\mathbf{x}]$ s.t. $k(\mathbf{x}, \mu) \in [a, b]$.

Therefore, we need to formulate another optimisation problem to solve for $\mu(\mathbf{x}) \in \sum[\mathbf{x}]$.

SOS Problem 2:

$$\begin{array}{ll} \mu(\mathbf{x}) = \arg \min_{\mu(\mathbf{x})} & \delta \\ \text{s.t.} & \Psi(\mathbf{x}, \mu) + \delta \in \sum[\mathbf{x}] \\ & k(\mathbf{x}, \mu) - a - s_0(\mathbf{x}) \cdot \psi(\mathbf{y}(\mathbf{x})) \in \sum[\mathbf{x}] \\ & b - k(\mathbf{x}, \mu) - s_1(\mathbf{x}) \cdot \psi(\mathbf{y}(\mathbf{x})) \in \sum[\mathbf{x}] \\ & \delta > 0 \\ & s_0(\mathbf{x}), s_1(\mathbf{x}) \in \sum[\mathbf{x}], \end{array} \quad (13)$$

where expression $k(\mathbf{x}, \mu)$ with $\mu(\mathbf{x})$ as decision variable is derived through the first SOS problem.

Since the result of the calculated control inputs $\mathbf{u} = k(\mathbf{x}, \mu)$ will be a rational polynomial, as the last step of (11) involves the inverse of the polynomial matrix $\mathbf{A}$, which takes the form

$$\frac{\begin{bmatrix} \text{poly} & \text{poly} \\ \text{poly} & \text{poly} \end{bmatrix}}{\det(\mathbf{A})} \begin{bmatrix} \text{poly} \\ \text{poly} \end{bmatrix}, \quad (14)$$

*‘poly’ represents some arbitrary polynomial expression.

where the result of $\det(\mathbf{A})$ is a polynomial itself, we need to convert the constraint (3) to:

$$a \cdot |\det(\mathbf{A})|^2 \leq k(\mathbf{x}) \cdot |\det(\mathbf{A})|^2 \leq b \cdot |\det(\mathbf{A})|^2, \quad (15)$$

as the SOS solver cannot deal with rational polynomials.

Similarly, since the polynomial $\mu(\mathbf{x})$ will appear in the denominator of $\Psi(\mathbf{x}, \mu)$, since $\mu(\mathbf{x}) \in \mathbf{R}^+$, to get rid of the denominator, the new constraint becomes

$$\Psi(\mathbf{x}, \mu) \cdot \mu \geq 0. \quad (16)$$

As a side note, all trigonometric functions need to be converted into polynomial form, for example via Taylor or Chebyshev expansion, since trigonometric functions themselves are generally not accepted by SOS solvers.

Thus, the reformulated SOS problem becomes

SOS Problem 2 (Reformulated):

$$\begin{aligned} \mu(\mathbf{x}) &= \arg \min_{\mu(\mathbf{x})} \delta \\ \text{s.t. } \Psi(\mathbf{x}, \mu) \cdot \mu + \delta &\in \sum [\mathbf{x}] \\ k(\mathbf{x}, \mu) \cdot |\det(\mathbf{A})|^2 - a \cdot |\det(\mathbf{A})|^2 - s_0(\mathbf{x}) \cdot \psi(\mathbf{y}(\mathbf{x})) &\in \sum [\mathbf{x}] \\ -\frac{\partial \mu(\mathbf{x})}{\partial \mathbf{x}} (f(\mathbf{x}) + g(\mathbf{x})k(\mathbf{x}, \mu)) &\in \sum [\mathbf{x}] \quad (\text{However, this may cause problem with quadratic constraint.}) \\ b \cdot |\det(\mathbf{A})|^2 - k(\mathbf{x}, \mu) \cdot |\det(\mathbf{A})|^2 - s_1(\mathbf{x}) \cdot \psi(\mathbf{y}(\mathbf{x})) &\in \sum [\mathbf{x}] \\ \delta &> 0 \\ s_0(\mathbf{x}), s_1(\mathbf{x}) &\in \sum [\mathbf{x}], \end{aligned} \quad (17)$$

where expression $k(\mathbf{x}, \mu)$ with $\mu(\mathbf{x})$ as decision variable is derived through the first SOS problem.

Note that, since both $\frac{\partial \mu(\mathbf{x})}{\partial \mathbf{x}}$ and $k(\mathbf{x}, \mu)$ will contain decision variables of μ , thus, this quadratic constraint may not be applicable for off-the-shelf SOS solvers.

Put in simple terms, one can simply use $\mu \in \mathbb{R}^+$ instead of $\mu(\mathbf{x}) \in \sum [\mathbf{x}]$, then the general algorithm becomes

SOS Problem 3 ($\mu \in \mathbb{R}^+$):

$$\begin{aligned} \mu &= \arg \min_{\mu} \delta \\ \text{s.t. } \Psi(\mathbf{x}, \mu) \cdot \mu + \delta &\in \sum [\mathbf{x}] \\ k(\mathbf{x}, \mu) \cdot |\det(\mathbf{A})|^2 - a \cdot |\det(\mathbf{A})|^2 - s_0(\mathbf{x}) \cdot \psi(\mathbf{y}(\mathbf{x})) &\in \sum [\mathbf{x}] \\ b \cdot |\det(\mathbf{A})|^2 - k(\mathbf{x}, \mu) \cdot |\det(\mathbf{A})|^2 - s_1(\mathbf{x}) \cdot \psi(\mathbf{y}(\mathbf{x})) &\in \sum [\mathbf{x}] \\ \mu &> 0 \\ \delta &> 0 \\ s_0(\mathbf{x}), s_1(\mathbf{x}) &\in \sum [\mathbf{x}], \end{aligned} \quad (18)$$

where expression $k(\mathbf{x}, \mu)$ with $\mu(\mathbf{x})$ as decision variable is derived through the first SOS problem.

3 Proof of validity

Given $\mathcal{C}$ defined by $\mathcal{C} = \{\mathbf{x} | \psi(\mathbf{y}(\mathbf{x})) > 0\}$ in the original proof, if $\mu(\mathbf{x})$ is a polynomial of $\mathbf{x}$, taking the derivative becomes

$$\dot{\Psi}(\mathbf{x}) = \sum_{i=1}^m \left[\frac{\partial \psi}{\partial y_i} \dot{y}_i - \sum_{l=1}^{\gamma_i-1} \frac{(\eta_{l+1}^i - k_l^i)^\top}{\mu_l^i} \left(\dot{\eta}_{l+1}^i - \sum_{s=1}^l \frac{\partial k_l^i}{\partial \eta_s^i} \dot{\eta}_s^i \right) + \sum_{l=1}^{\gamma_i-1} \frac{\dot{\mu}_l^i}{2 (\mu_l^i)^2} \|\eta_{l+1}^i - k_l^i\|_2^2 \right]. \quad (19)$$

For each component $i \in \{1, \dots, m\}$, by incorporating k_l^i and arranging the corresponding terms

$$\begin{aligned}
& \frac{\partial \psi}{\partial y_i} \dot{y}_i - \sum_{l=1}^{\gamma_i-1} \frac{(\eta_{l+1}^i - k_l^i)^\top}{\mu_l^i} \left(\dot{\eta}_{l+1}^i - \sum_{s=1}^l \frac{\partial k_l^i}{\partial \eta_s^i} \dot{\eta}_s^i \right) + \sum_{l=1}^{\gamma_i-1} \frac{\dot{\mu}_l^i}{2(\mu_l^i)^2} \|\eta_{l+1}^i - k_l^i\|_2^2 \\
&= \frac{\partial \psi}{\partial y_i} (\eta_2^i - k_1^i + k_1^i) \\
&\quad - \sum_{l=1}^{\gamma_i-2} \frac{(\eta_{l+1}^i - k_l^i)^\top}{\mu_l^i} \left(\dot{\eta}_{l+1}^i - k_l^i + k_l^i - \sum_{s=1}^l \frac{\partial k_l^i}{\partial \eta_s^i} \dot{\eta}_s^i \right) + \sum_{l=1}^{\gamma_i-2} \frac{\dot{\mu}_l^i}{2(\mu_l^i)^2} \|\eta_{l+1}^i - k_l^i\|_2^2 \\
&\quad - \frac{(\eta_{\gamma_i}^i - k_{\gamma_i-1}^i)^\top}{\mu_{\gamma_i-1}^i} \left[\dot{\eta}_{\gamma_i}^i - \sum_{s=1}^{\gamma_i-1} \frac{\partial k_{\gamma_i-1}^i}{\partial \eta_s^i} \dot{\eta}_s^i \right] + \frac{\dot{\mu}_{\gamma_i-1}^i}{2(\mu_{\gamma_i-1}^i)^2} \|\eta_{\gamma_i}^i - k_{\gamma_i-1}^i\|_2^2 \\
&= \frac{\partial \psi}{\partial y_i} k_1^i - (\eta_2^i - k_1^i)^\top \underbrace{\left[-\frac{\partial \psi}{\partial y_i} + \frac{k_2^i - \sum_{s=1}^1 \frac{\partial k_1^i}{\partial \eta_s^i} \dot{\eta}_s^i}{\mu_1^i} \right]}_{\mathcal{I}_1} + \frac{\dot{\mu}_1^i}{2(\mu_1^i)^2} \|\eta_2^i - k_1^i\|_2^2 \\
&\quad - (\eta_3^i - k_2^i)^\top \underbrace{\left[\frac{\eta_2^i - k_1^i}{\mu_1^i} + \frac{k_3^i - \sum_{s=1}^2 \frac{\partial k_2^i}{\partial \eta_s^i} \dot{\eta}_s^i}{\mu_2^i} \right]}_{\mathcal{I}_2} + \frac{\dot{\mu}_2^i}{2(\mu_2^i)^2} \|\eta_3^i - k_2^i\|_2^2 \\
&\quad \dots \\
&\quad - (\eta_{\gamma_i}^i - k_{\gamma_i-1}^i)^\top \underbrace{\left[\frac{\eta_{\gamma_i-1}^i - k_{\gamma_i-2}^i}{\mu_{\gamma_i-2}^i} + \frac{\dot{\eta}_{\gamma_i}^i - \sum_{s=1}^{\gamma_i-1} \frac{\partial k_{\gamma_i-1}^i}{\partial \eta_s^i} \dot{\eta}_s^i}{\mu_{\gamma_i-1}^i} \right]}_{\mathcal{I}_{\gamma_i-1}} + \frac{\dot{\mu}_{\gamma_i-1}^i}{2(\mu_{\gamma_i-1}^i)^2} \|\eta_{\gamma_i}^i - k_{\gamma_i-1}^i\|_2^2.
\end{aligned} \tag{20}$$

In order to guarantee

$$\begin{aligned}
\dot{\Psi}(\mathbf{x}) &= \frac{\partial \psi}{\partial \mathbf{y}} \mathbf{k}_1(\mathbf{y}(\mathbf{x})) - \sum_{i=1}^m \sum_{l=1}^{\gamma_i-1} \frac{\lambda}{2\mu_l^i} \|\eta_{l+1}^i - k_l^i(\mathbf{z}_l)\|_2^2 \\
&\geq \lambda \psi(\mathbf{y}(\mathbf{x})) - \sum_{i=1}^m \sum_{l=1}^{\gamma_i-1} \frac{\lambda}{2\mu_l^i} \|\eta_{l+1}^i - k_l^i(\mathbf{z}_l^i)\|_2^2 \\
&= \lambda \Psi(\mathbf{x}),
\end{aligned} \tag{21}$$

it is needed to further guarantee that the newly added blue terms are no greater than zero, i.e. $\dot{\mu}_l^i \leq 0, l = \{1, \dots, \gamma_i - 1\}$.

Thus, another constraint should also be enforced

$$\frac{\partial \mu_l^i}{\partial \mathbf{x}} (f(\mathbf{x}) + g(\mathbf{x})k(\mathbf{x})) \leq 0. \tag{22}$$

3.1 Some Julia & Python tricks

When doing Julia calls in Python, we cannot directly pass Python expressions into Julia. One trick being used is to parse the Julia expression into a String, and then convert the Julia code string to an executable expression:

Listing 1: Convert Julia string to executable expression

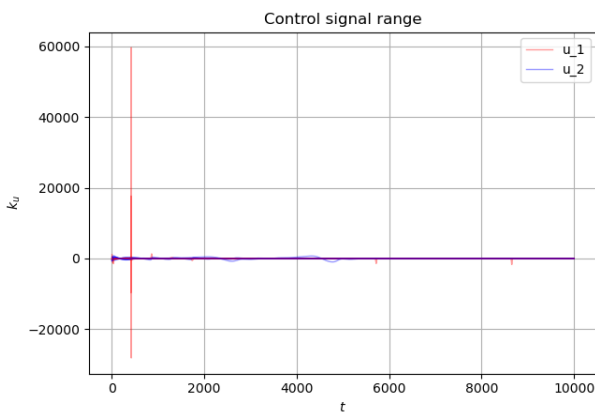
```

function str2expr(expr_str::String)
    parsed_expr = Meta.parse(expr_str)
    return parsed_expr
end

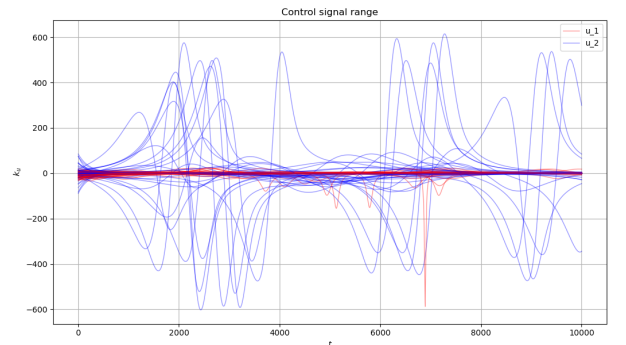
```

4 Tested results

A sample test result can be shown in Fig.1, with the y -axis as the magnitude of the control inputs being significantly constrained.



(a) Original control inputs for dubins car



(b) Bounded control inputs for dubins car

Figure 1: Comparison between the original and after incorporating bounds for control inputs